Language Modeling on Penn Treebank

JOU-YI LEE
UC Santa Cruz, M.S. in NLP
65 Rio Robles E
San Jose, CA 95134
jlee1039@ucsc.edu

Abstract

Using self-supervised techniques, this project explores language modeling on the Penn Treebank dataset with the goal of predicting the subsequent token in a sequence based on previous tokens.

I develop and assess many neural network topologies, such as Recurrent Neural Networks (RNN), Long Short-Term Memory Networks (LSTM), and Gated Recurrent Units (GRU), due to the modest dataset size (about 1 million tokens) in comparison to contemporary norms.

Without depending on any pretrained embeddings or external libraries for model generation or training procedures, models are trained using randomly initialized embeddings to uniquely adapt to the Penn Treebank corpus.

In this project, I build and evaluate a language model using the Penn Treebank dataset. Perplexity is calculated as the exponentiated average negative log-probability of the predicted tokens, formulated as:

$$\text{Perplexity} = \exp\left(-\frac{1}{T}\sum_{i=1}^{T}\log p(t_i|t_{<i})\right)$$

where T is the total number of tokens and $p(t_i|t_{<i})$ represents the conditional probability of each token t_i given its preceding tokens. Lower perplexity values indicate better predictive performance.

Introduction

With the express goal of predicting the next token in a sequence given the preceding tokens, we are developing a GRU language model in this project to carry out self-supervised learning on the Penn Treebank dataset.

Since language modeling makes possible applications like text generation, speech recognition, and machine translation, it is a crucial problem in natural language processing. Using perplexity as the primary performance metric, this study aims to assess several model architectures on the Penn Treebank dataset.

The data is preprocessed through tokenization and vocabulary building, converting each word to its corresponding index based on a custom vocabulary. An example sentence from the dataset is:

> mr. <unk> is chairman of <unk> n.v. the dutch publishing group

> aer banknote berlitz calloway centrust cluett fromstein gitano guterman hydroquebec ipo kia memotec mlx nahb punts rake regatta rubens sim snack-food ssangyong swapo wachter

1 Methodology

The dataset is tokenized and each token is encoded into a series of indices in order to apply the model. To learn the probability distribution over the subsequent token, the training, validation, and test splits are processed and input into a model based on a recurrent neural network, specifically a GRU with dropout.

Perplexity, a measure of the model's ability to generalize to new data, is used to assess its performance. In order to meet the objectives of the assignment, the complete training and assessment pipeline is constructed without the use of external libraries for model generation.

Top 20 Most Common Tokens

This bar chart (Figure 1) displays the frequency of the dataset's 20 most common tokens (words). The word "the" is the most frequent, followed by "<UNK>" (which represents unknown or out-of-vocabulary tokens).

Other common words include "of," "to," "a," and "in," meaning they are frequently used in English but generally carry less semantic meaning.

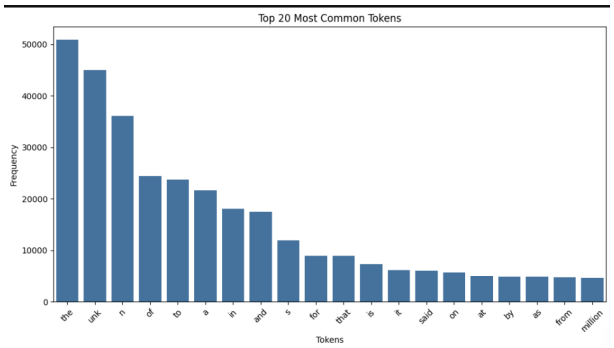

Figure 1: Top 20 Most Common Tokenss

Sentence Length Distribution

This histogram (Figure 2) shows the distribution of sentence lengths based on the number of tokens. The peak is around 20 tokens, indicating that most sentences in the dataset contain roughly 20 words. Very few sentences have more than 60 tokens, which shows that sentences are relatively short.

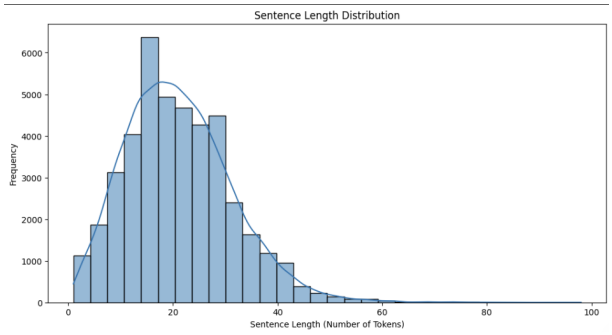

Figure 2: Sentence Length Distribution

Word Cloud of Tokens in the Dataset

Common English words like “the,” “and,” “to,” “for,” and “of” appear prominently. Some content words, such as “company,” “market,” “million,” and “shares,” also appear, suggesting that the dataset may have a financial or business-related theme.

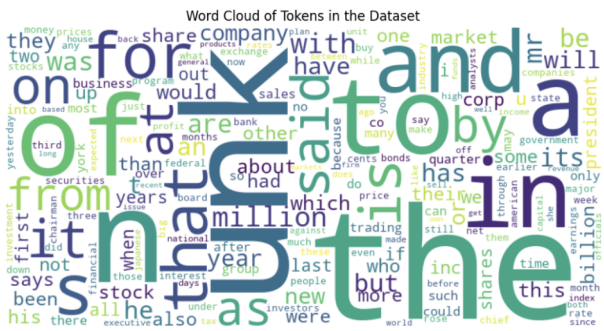

Figure 3: Word Cloud of Tokens

1.1 Handling Data Sparsity and Imbalance

The PTB dataset contains a high frequency of common words (e.g., "the," "of") and many occurrences of the <UNK> token for unknown words, resulting in data sparsity for certain rare words. To address this:

- We employed a vocabulary threshold, filtering out rare words that appeared only once or twice in the dataset. This helps in focusing the model on more frequent and meaningful words.
- Dropout regularization was applied to prevent over-reliance on certain words and to improve generalization.
- The dataset split was stratified to ensure a balanced distribution of common and uncommon words across the training, validation, and test sets.

2 model structure

The model is structured as follows:

- **Embedding Layer**: A custom embedding layer maps each word in the vocabulary to a 100-dimensional vector. These embeddings are randomly initialized and trained along with the model to adapt to the specific data distribution in the Penn Treebank dataset.
- **GRU Layers**: A GRU with up to 3 layers (as determined by Optuna’s hyperparameter tuning) is used, with the number of hidden units ranging from 64 to 512. The GRU layers capture sequential dependencies, aiding in effective language modeling.
- **Dropout Layer**: Dropout with a probability between 0.1 and 0.7 is applied to prevent overfitting and improve generalization. The dropout rate is also optimized using Optuna.
- **Fully Connected Layer**: A final fully connected layer maps the GRU output to the vocabulary size, allowing the model to predict the probability of each word as the next token in the sequence.

2.1 Data Preprocessing

Data preparation includes vocabulary creation and tokenization. Each sentence is changed to lowercase during tokenization, and the text is divided

using a regular expression so that only alphanumeric substrings remain. The text becomes a list of words as a result of this process.

Next, we build a vocabulary by counting the occurrences of each word in the entire dataset and assigning a unique index to each word. To address the issue of inconsistent sequence lengths, a <PAD> token is added to ensure uniform sequence lengths.

The `encode_text` function is used to convert each word in a sentence to its corresponding index based on the vocabulary. Finally, we utilize a `DataLoader` to load the data, dividing each sequence into inputs and targets. This approach sets the inputs as the first $n-1$ words of the sequence and the targets as the last $n-1$ words, facilitating the model in learning the relationship between each word and the subsequent word in the sequence.

2.2 Embedding Methods

For this project, I put every word in the vocabulary as a fixed-dimensional vector using a custom embedding layer. The Penn Treebank dataset is used to jointly train the model and initialize the embedding layer at random. The model is able to capture the semantic associations between words as it learns because each word is mapped to a 100-dimensional vector.

For language modeling, the embedding layer provides the GRU model an important input that helps it better capture the context of each word in a sentence. To ensure constant sequence lengths, padding is inserted as necessary using the unique <PAD> token. For effective batch processing and model training, this guarantees that every sequence in a batch is the same size.

Unlike pre-trained embeddings such as Word2Vec or GloVe, this embedding layer is specifically tailored to the Penn Treebank dataset, as the assignment guidelines require.

2.3 Dataset Splitting

The dataset used in our experiments is the Penn Treebank (PTB) text-only dataset. We split this dataset into three subsets:

- **Training Set**: Used for training the models. This set constitutes 80% of the full dataset.
- **Validation Set**: Used for tuning hyperparameters and assessing model performance during training. This set constitutes 10% of the full dataset.
- **Test Set**: Used to evaluate the final model's performance. This set constitutes 10% of the full dataset and was not used during the training or hyperparameter tuning phases.

The splits were done to ensure a balanced distribution of text data across each subset, maintaining similar word distributions.

3 Model: GRU Language Model with Embedding Layer(Basic)

The language model for this task is a Gated Recurrent Unit (GRU) network, designed to process sequential data and predict the next token in a sequence. The GRU layer has two layers, which can capture long-term dependencies in the token sequences. The architecture consists of the following components:

- **Embedding Layer**: The embedding layer maps each word in the vocabulary to a 100-dimensional vector, allowing the model to learn semantic relationships between words. The embeddings are randomly initialized and trained jointly with the model on the Penn Treebank dataset.
- **GRU Layer**: The GRU layer has two layers with 256 hidden units each, enabling the model to capture dependencies across tokens in the sequence effectively.
- **Dropout Layer**: A dropout layer with a probability of 0.5 is applied after the GRU layer to reduce overfitting and enhance generalization.
- **Fully Connected Layer**: A fully connected (FC) layer maps the GRU output to the vocabulary size, predicting the probability of each token in the vocabulary as the next token in the sequence.

The model uses cross-entropy loss with padding tokens ignored, and AdamW as the optimizer with a learning rate of 0.001. A learning rate scheduler, `ReduceLROnPlateau`, is applied to decrease the learning rate by a factor of 0.5 if the validation loss does not improve over three epochs, helping to achieve optimal performance.

3.1 Model Selection for Test Set Submission

After running this configuration, I evaluated it on the test set. The perplexity score on the validation set was 245.63. After submitting to **Kaggle**, the

perplexity score on the test set was 267.68, which was used for the first Kaggle submission.

4 Experiments Implementation Based on Optuna

In this Project I apply Optuna for automatic tuning in order to maximize the model's hyperparameters. The number of layers, learning rate, dropout probability, batch size, weight decay, and the number of hidden units in the GRU are the main hyperparameters. The validation perplexity determines how these parameters are changed. AdamW is used as the optimizer and cross-entropy loss is used to train the model, disregarding padding tokens. In order to avoid overfitting and conserve computational resources, an early stopping mechanism is used, which stops training if validation perplexity does not improve over a predetermined number of epochs.

4.1 Setting Optuna Hyperparameters

- **Hidden Dimension (hidden_dim)**: The GRU hidden dimension was sampled between 64 and 512. This controls the size of the model's hidden states, balancing model capacity and computational cost.

- **Number of Layers (num_layers)**: The number of GRU layers was chosen from 1, 2, or 3. Having multiple layers provides sufficient depth for capturing complex patterns in sequential data.

- **Learning Rate (learning_rate)**: The learning rate was sampled from a log-uniform distribution between 1×10^{-4} and 1×10^{-2}. This controls the step size in gradient descent and plays a crucial role in achieving optimal convergence.

- **Batch Size (batch_size)**: Batch size was selected from 16, 32, or 64. It influences the number of samples processed per gradient update, impacting both training stability and computational efficiency.

- **Dropout Probability (dropout_prob)**: The dropout probability, used to mitigate overfitting, was sampled uniformly between 0.1 and 0.7. Dropout helps prevent the model from over-relying on certain nodes, thus enhancing generalization.

- **Weight Decay (weight_decay)**: The weight decay was sampled from a log-uniform distribution between 1×10^{-6} and 1×10^{-2}, acting as an L2 regularization term. This helps prevent overfitting by penalizing large weights, thus encouraging simpler models with better generalization.

Each trial in the Optuna study was evaluated based on the validation perplexity, which served as the primary metric for selecting the best hyperparameters. Early stopping was also applied to prevent overfitting and reduce unnecessary computation for configurations that did not perform well.

4.2 GRU Language Model Configurations and Hyperparameter Testing

For the GRU-based language model, we tested various configurations:

- **GRU Layers**: We experimented with 1, 2, and 3 layers to observe the effect of model depth on perplexity.

- **Hidden Dimension**: Tested values ranged from 64 to 512 to evaluate the effect on model capacity and learning.

- **Dropout and Weight Decay**: Dropout probabilities from 0.1 to 0.7 and weight decay values from 0.000001 to 0.01 were tested to find the optimal regularization level for minimizing overfitting.

Each configuration was evaluated using validation perplexity as the main criterion. The best configuration was identified through Optuna and further verified by retraining and evaluating on the test set.

5 Results

5.1 Evaluation and Selection of Final Model

To evaluate model performance, we used perplexity as the primary metric, calculated as:

$$\text{Perplexity} = \exp\left(-\frac{1}{T}\sum_{i=1}^{T}\log p(t_i|t_{<i})\right)$$

where T is the total number of tokens in a sentence. A lower perplexity indicates a better model, as it means the model is more confident in its predictions.

5.2 Model Selection for Test Set Submission

After running multiple trials with Optuna, the best-performing model configuration was selected based on the lowest validation perplexity. This configuration was then retrained on the entire training set (using the chosen hyperparameters) and evaluated on the test set. The perplexity score from the test set was used for the final Kaggle submission.

5.2.1 Best Hyperparameters

The best hyperparameters obtained from the tuning process are as follows:

- **Hidden Dimension (hidden_dim)**:
- **Number of Layers (num_layers)**:
- **Learning Rate (learning_rate)**:
- **Batch Size (batch_size)**:
- **Dropout Probability (dropout_prob)**:
- **Weight Decay (weight_decay)**:

5.3 Performance on Training, Validation, and Test Sets

Table 1 summarizes the performance on training and validation sets for different hyperparameter configurations.

Trial	Hidden Dim	Layers	Learning Rate	Batch Size	Dropout	Val. Perplexity
1	256	2	0.001	32	0.3	270.32
2	128	1	0.0005	64	0.4	260.45
⋮	⋮	⋮	⋮	⋮	⋮	⋮
Best	451	2	0.0003	64	0.36	245.69

Table 1: Optuna Trials and Validation Perplexities

5.4 Optimization History

Figure 4 shows the trend of objective values across different trials. The red line represents the current best perplexity, while the blue dots indicate the perplexity values for each trial. As the number of trials increases, the perplexity gradually decreases and stabilizes, demonstrating the improvement in Optuna's optimization performance.

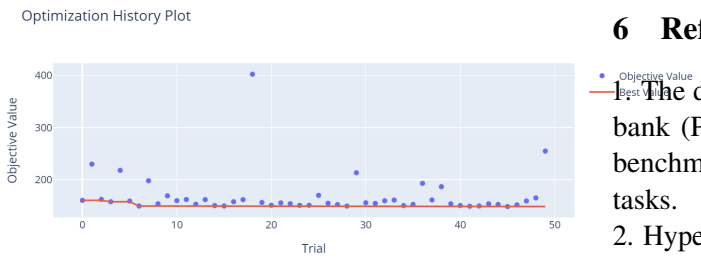

Figure 4: Objective Value Across Trials

5.5 Parallel Coordinate Plot

Figure 5 illustrates the impact of different hyperparameter combinations on perplexity in each trial. Parameters such as learning rate and hidden dimension show a significant influence on the final perplexity.

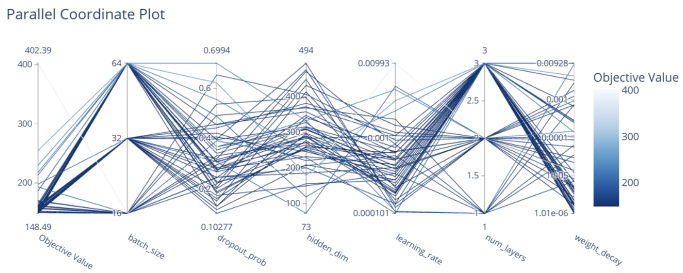

Figure 5: Parallel Coordinate Plot of Hyperparameters and Perplexity

5.6 Influence of Hyperparameters

Figure 6 shows the impact of each hyperparameter on model performance, as generated by Optuna.

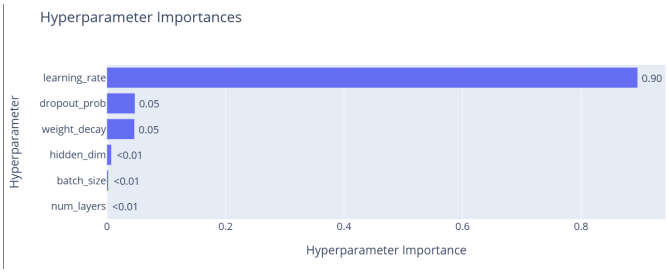

Figure 6: Hyperparameter Importance in Validation Perplexity

5.7 Comparison with Related Work

Our findings compare previous studies (No hyperparameter adjustment), showing that increasing the hidden dimension and using dropout improves model robustness. The best configuration achieved a validation perplexity of 148.49, and the final test on textbfKaggle perplexity was 245.69.

6 References

1. The dataset used in this project is the Penn Treebank (PTB) dataset, which is a commonly used benchmark dataset in natural language processing tasks.
2. Hyperparameter optimization was performed using Optuna, an automatic hyperparameter optimization framework. Optuna provides an efficient way

to tune model parameters through various strategies, including Bayesian optimization.